Precision Voltage Glitching Using FPGAs and Emulations

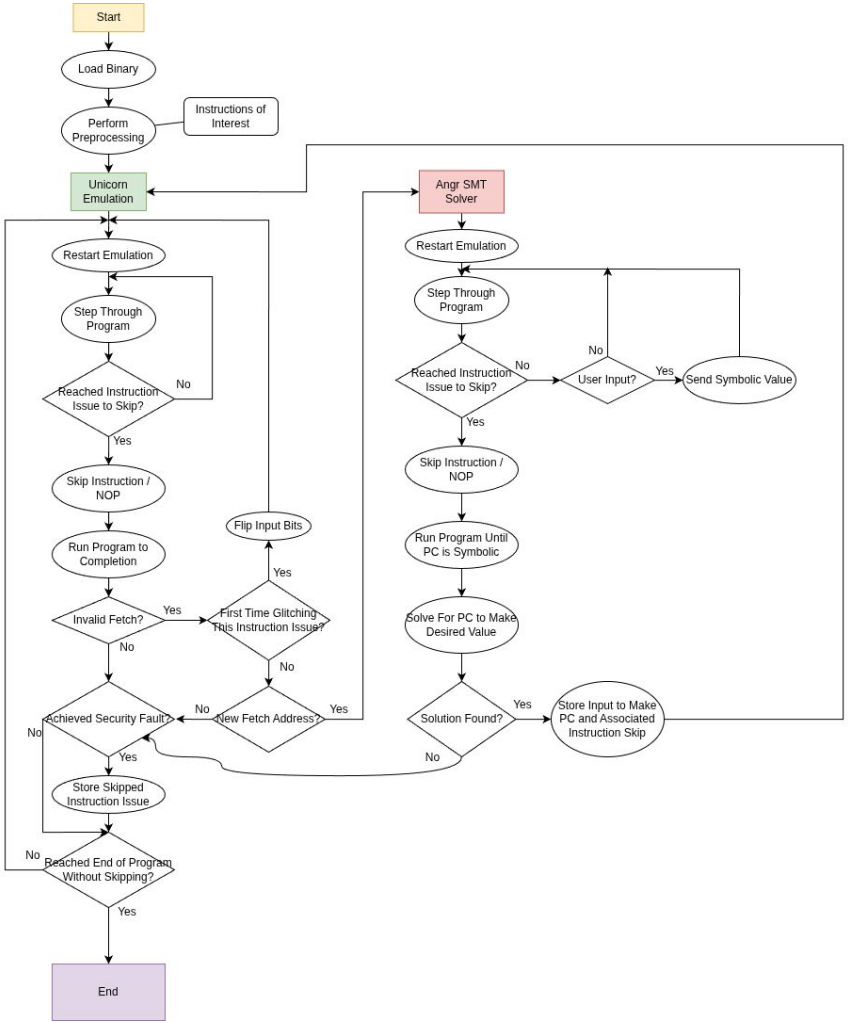
Geeoon Chung and Nate Snyder

Introduction

- Physical access gives us a new attack vector
  - Even perfect software can now be hacked
- Voltage glitching
  - Cause sags in the voltage
  - Signal propagation through a combination circuit is a function of the voltage supplied
  - Effectively cause timing violations between flip flops / pipeline stages
    - Precise timing allows us to corrupt certain instructions through a CPU pipeline
      - Most commonly a duplicated instruction or a NOP
- We present a new way to reliably find and glitch instructions
- We also present a new way of gaining arbitrary code execution

Our Software: Finding Glitchable Instructions

- Preprocessing using Capstone
  - Find address of instructions that might be interesting to skip
- Emulate in Unicorn
  - Skip the instruction at the nth instruction execution then run until completion
  - Did this result in the expected output for a security fault (e.g. stdout prints “access granted”)
  - Did this result in an invalid fetch? Does changing the program input result in an invalid fetch at a different address?
    - Our input has some influence on the program counter
  - Use the preprocessing to reduce search space significantly
- Emulate in Angr
  - Only ran if the Unicorn emulation indicated control of the program counter
  - SMT Solver
    - Give symbolic inputs
    - Figure out which input will give us a desired PC
  - Much slower than Unicorn, so use sparingly


Start
Load Binary
Perform Preprocessing
Instructions of Interest
Unicorn Emulation
Restart Emulation
Step Through Program
Reached Instruction Issue to Skip?
No
Yes
Skip Instruction / NOP
Run Program to Completion
Invalid Fetch?
Yes
No
First Time Glitching This Instruction Issue?
Yes
No
Flip Input Bits
New Fetch Address?
No
Yes
Achieved Security Fault?
No
Yes
Store Skipped Instruction Issue
Reached End of Program Without Skipping?
No
Yes
End
Angr SMT Solver
Restart Emulation
Step Through Program
Reached Instruction Issue to Skip?
No
User Input?
No
Yes
Send Symbolic Value
Yes
Skip Instruction / NOP
Run Program Until PC is Symbolic
Solve For PC to Make Desired Value
Solution Found?
Yes
Store Input to Make PC and Associated Instruction Skip
No

Regular Glitching

- Some instruction skips cause branches to be taken/not taken
- Example: Password Check

```c
#include "string.h"
#define PASSWORD "password123"

extern void init_device(void);
extern int _write(int fd, char* buf, int len);
extern int _read(int fd, char* buf, int len);
extern void pwned(void);

int main() {
    init_device();
    volatile int lol = 0;
    if (lol) pwned();
    char input[100];
    do {
        _write(0, "enter a password:", strlen("enter a password:"));
        int n = _read(0, input, strlen(PASSWORD) + 1);
        input[n - 1] = '\0';
    } while(strncmp(input, PASSWORD, strlen(PASSWORD)) != 0);
    _write(0, "access granted.", strlen("access granted."));
    return 0;
}
```

```
  b4:   0010            movs    r0, r2
  b6:   b530            push    {r4, r5, lr}
  b8:   2a00            cmp     r2, #0
  ba:   dd06            ble.n   0xca
  bc:   4c03            ldr     r4, [pc, #12]   @ (0xcc)
  be:   188d            adds    r5, r1, r2
  c0:   780b            ldrb    r3, [r1, #0]
  c2:   3101            adds    r1, #1
  c4:   7023            strb    r3, [r4, #0]
  c6:   42a9            cmp     r1, r5
```

```
Fault target address: 0x10000be
Fault instruction issue #: 192

Instruction(s):
  0x10000be: adds r5, r1, r2
Exit Code:
  None
Input to program:
  b'a\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00'
Output:
  b'enter a password:enter a password:\x00access granted.\x00\x00\x00\x00\x00\x0
0\x00\x00\x00'
```

Controlling the Program Counter

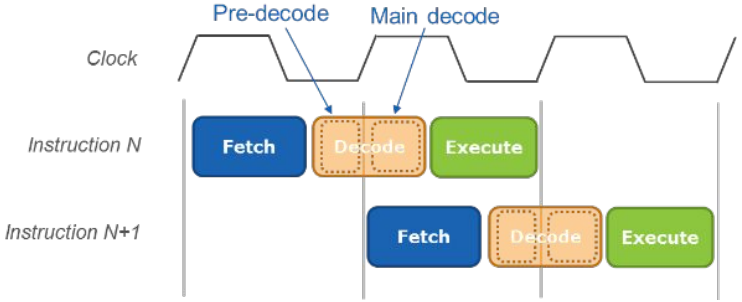
- Instructions like `pop pc` will put stack data into the program counter
- If an instruction skip corrupted the stack with user controlled data, we can jump to any instruction
- Example: AES ECB (symmetric encryption/decryption)

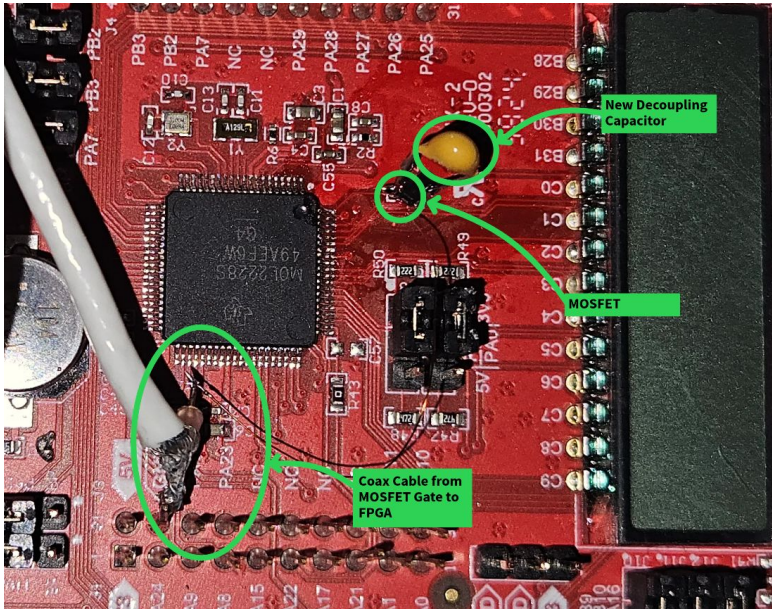
```c
#include "AES_256_ECB.h"

extern void init_device(void);
extern int _read(int fd, char* buf, int len);
extern void pwned(void);

int main(void) {
    init_device();
    volatile int dummy = 0;
    if (dummy) pwned();

    AES_CTX ctx;
    const unsigned char key[AES_KEY_SIZE] = { 0xe7, 0x7d, 0xbb, 0xa2, 0x30, 0x9c, 0xdc, 0xa3 };
    unsigned char data[AES_BLOCK_SIZE];

    // get data
    int n = _read(0, data, AES_BLOCK_SIZE);

    // encrypt data
    AES_EncryptInit(&ctx, key);
    AES_Encrypt(&ctx, data, data);

    // decrypt data
    AES_DecryptInit(&ctx, key);
    AES_Decrypt(&ctx, data, data);

    return 0;
}
```

```
(.venv) main@debian ~/D/s/Fault-Injection-Finder (main)> python3 main.py ./targe
ts/timspm0l2228/source/example/aes_ecb.bin ./inputs/aes_ecb.bin -i 1000 -d 0x000
01b74
Fault Injection Finder

WARNING  | 2026-05-26 01:00:04,602 | angr.engines.successors | Exit state has ov
er 256 possible solutions. Likely unconstrained; skipping. <BV32 io_read_49_8 ..
 io_read_48_8 .. io_read_47_8 .. io_read_46_8>
WARNING  | 2026-05-26 01:00:09,508 | angr.engines.successors | Exit state has ov
er 256 possible solutions. Likely unconstrained; skipping. <BV32 io_read_115_8 .
. io_read_114_8 .. io_read_113_8 .. io_read_112_8>
Found 2 faults.
==================================================
Fault target address: 0x100035e
Fault instruction issue #: 17

Instruction(s):
  0x100035e: push {r4, r5, lr}

Exit Code:
  None

Input to program:
  b'\xce\xce\xce\xce\xce\xce\xce\xce\xce\xce\xce\xce\xce\xce\xce\xce'

Output:
  b''
Got control of the PC
!!!!!!!!!!
By giving an input of b'\x00\x00\x00\x00u\x1b\x00\x00\x00\x00\x00\x00\x00\x00\x0
0\x00', we get the desired PC value of 0x1b74
!!!!!!!!!!
```

Our Hardware

- TI-MSPM0L2228
  - Arm Cortex-M0+
    - Two Stage Pipeline
  - No cache
  - 32 MHz
  - Predictable Cycles Per Instruction
- Remove decoupling capacitor and replace with smaller capacitor
- Add a MOSFET, Si2302
  - Temporarily connect V_{CORE} to ground
  - Gate controlled by FPGA





FPGA Glitching

- GPIO for attacking specific instructions
  - The program calculates the target, the FPGA executes the glitch
  - Start when a trigger signal from the board is received
- A PLL (phase-locked loop) multiplies the clock frequency
  - The PLL takes the effective clock frequency from 50 MHz to 400 MHz
  - Way faster than the target board! Running at 32 MHz
- Precision is still hard
  - Somewhat trial and error
  - Aim for different instructions using different glitch lengths
- Automated communication
  - UART sends packets back-and-forth between the FPGA and program

Results / Live Demo

Real World Impact / Use Cases

- Hacking into phones
- Hacking into military equipment
- Hacking into secured buildings
- Hacking into computers
- Hacking into consoles
- Anything where physical access is involved is vulnerable

'Worth it': FBI admits it paid $1.3m to hack into San Bernardino iPhone

The hefty price paid for the software that hacked Syed Farook's iPhone, which Apple refused to help the FBI break into, signals a growing 'exploit market'

Microsoft's 'unhackable' Xbox One has been hacked by 'Bliss' — the 2013 console finally fell to voltage glitching, allowing the loading of unsigned code at every level

News By Mark Tyson published March 15, 2026

This console had remained a fortress since its launch over a decade ago.

fTPM Voltage Fault Injection

Bulletin ID: AMD-SB-4005
Potential Impact: Arbitrary Code Execution
Severity: High

Summary

CVE-2023-20589

Researchers at the Technische Universität Berlin have reported the use of voltage fault injection attacks on ASP secure boot targeting fTPM. An attacker with specialized hardware and physical access to an impacted device may be able to perform a voltage fault injection attack resulting in compromise of the ASP secure boot.

Questions?

GitHub

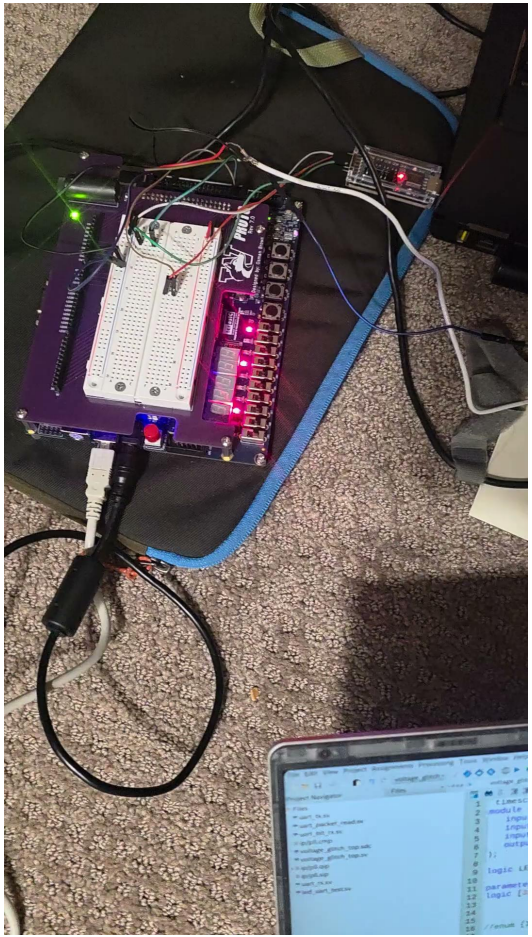
https://github.com/Geeoon/Fault-Injection-Finder

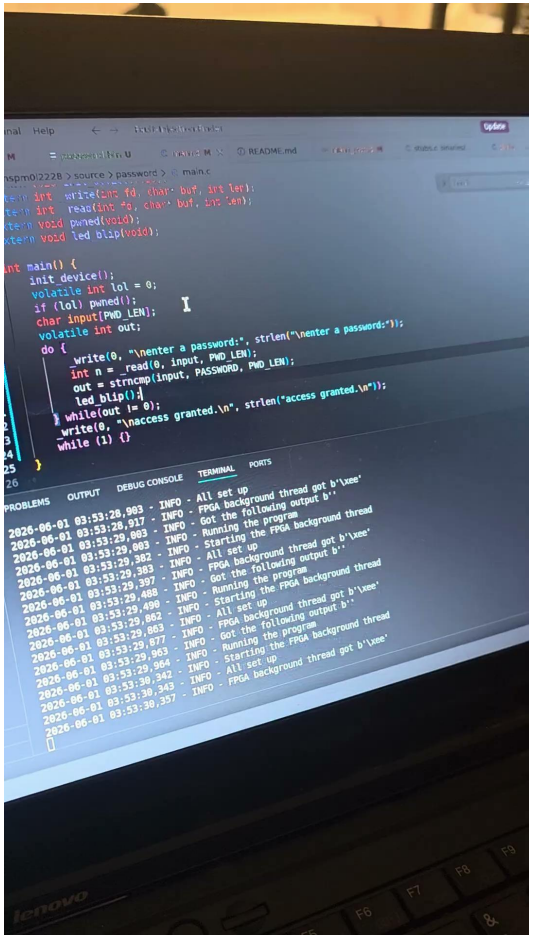
https://github.com/Ice-Skates/voltage_glitch

References

https://developerhelp.microchip.com/xwiki/bin/view/products/mcu-mpu/32bit-mcu/sam/arm-cortex-m0/pipeline/

Demo failed :(




int write(int fd, char* buf, int len);
int read(int fd, char* buf, int len);
void pwned(void);
void led_blip(void);
int main() {
init_device();
volatile int lol = 0;
if (lol) pwned();
char input[PWD_LEN];
volatile int out;
do {
_write(0, "\nenter a password:", strlen("\nenter a password:"));
int n = _read(0, input, PWD_LEN);
out = strncmp(input, PASSWORD, PWD_LEN);
led_blip();
} while(out != 0);
_write(0, "\naccess granted.\n", strlen("access granted.\n"));
while (1) {}
}
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
2026-06-01 03:53:28,903 - INFO - All set up
2026-06-01 03:53:28,917 - INFO - FPGA background thread got b'\xee'
2026-06-01 03:53:29,003 - INFO - Got the following output b''
2026-06-01 03:53:29,003 - INFO - Running the program
2026-06-01 03:53:29,382 - INFO - Starting the FPGA background thread
2026-06-01 03:53:29,383 - INFO - All set up
2026-06-01 03:53:29,397 - INFO - FPGA background thread got b'\xee'
2026-06-01 03:53:29,488 - INFO - Got the following output b''
2026-06-01 03:53:29,490 - INFO - Running the program
2026-06-01 03:53:29,862 - INFO - Starting the FPGA background thread
2026-06-01 03:53:29,863 - INFO - All set up
2026-06-01 03:53:29,877 - INFO - FPGA background thread got b'\xee'
2026-06-01 03:53:29,963 - INFO - Got the following output b''
2026-06-01 03:53:29,964 - INFO - Running the program
2026-06-01 03:53:30,342 - INFO - Starting the FPGA background thread
2026-06-01 03:53:30,343 - INFO - All set up
2026-06-01 03:53:30,357 - INFO - FPGA background thread got b'\xee'
2026-06-01 03:53:30,
lenovo